Yellowbrick

Reusable Business Logic in Yellowbrick

Module 8: Reusable Business Logic

Purpose: Learn three tools for encoding reusable logic in Yellowbrick: Views, Stored Procedures, and Python. Understand when to use each and how they work together.

Main Topics: Views · Stored Procedures · YB SP limitations · SAS-to-SQL patterns · YBSQL · Python analytics with psycopg2

Audience: Business analysts and data professionals migrating from SAS or building reusable SQL logic in Yellowbrick

What is a View?

A named, saved SELECT query. No data stored — Yellowbrick re-executes the underlying query each time.

Why views matter

- Write complex joins once — anyone can query the view without knowing the underlying tables
- Encode business rules centrally — one definition, consistent results for all analysts
- Control access — GRANT SELECT on the view, not on each underlying table
- Always returns fresh data — no stale snapshots, no scheduling needed
- Drop-in replacement — query a view exactly like a regular table

Basic syntax

```
-- Create a view
CREATE OR REPLACE VIEW
    public.view_name AS
SELECT col1, col2, ...
FROM   base_table
JOIN   other_table USING (key);

-- Query it like a table
SELECT * FROM public.view_name
WHERE  column = 'value';

-- Update: just re-run CREATE
CREATE OR REPLACE VIEW ...

-- Remove a view
DROP VIEW public.view_name;
```

Tip

Views are 'late-bound' — references resolve at query time. A view can reference tables in other databases. Update the view SQL and all users get the fix automatically.

View Example 1: Simplifying Joins

Problem: every analyst writes the same 5-table join. Solution: encode it once as vw_sales_detail.

Without a view — repeated, error-prone

```
-- Every analyst writes this manually:
SELECT
  f.transaction_id,
  d.full_date          AS sale_date,
  d.month_name, d.year,
  s.store_name, s.city, s.state,
  c.first_name||' '||c.last_name
                       AS customer_name,
  c.membership_tier,
  p.name               AS product_name,
  p.category,
  f.quantity, f.unit_price,
  f.discount_amount, f.total
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_date d
  ON f.date_key    = d.date_key
JOIN samples.retail.dim_store s
  ON f.store_key   = s.store_key
JOIN samples.retail.dim_customer c
  ON f.customer_key= c.customer_key
JOIN samples.retail.dim_product p
  ON f.product_key = p.product_key
```

With vw_sales_detail — write once, query anywhere

1 Define once in your schema

CREATE OR REPLACE VIEW public.vw_sales_detail AS SELECT ... — five tables joined, columns renamed, done.

2 Query it like a table

SELECT category, SUM(total) FROM public.vw_sales_detail WHERE year=2026 GROUP BY category;

3 Business rule in one place

Column renames, join conditions defined once. Update the view — every downstream query benefits.

4 Access control simplified

GRANT SELECT ON public.vw_sales_detail TO analyst_role; — not on each of the 5 source tables.

View Example 2: Encoding Business Logic

vw_customer_spend — pre-aggregated customer metrics

```
CREATE OR REPLACE VIEW
    public.vw_customer_spend AS
SELECT
    c.customer_key,
    c.customer_id,
    c.first_name||' '||c.last_name
                AS customer_name,
    c.membership_tier,
    COUNT(DISTINCT f.transaction_id)
                AS transaction_count,
    ROUND(SUM(f.total), 2)
                AS total_spend,
    ROUND(AVG(f.total), 2)
                AS avg_basket_size,
    MAX(f.transaction_timestamp)
                AS last_purchase_date
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_customer c
  ON f.customer_key = c.customer_key
GROUP BY 1, 2, 3, 4
```

Query the view — no aggregation needed

```
-- Query Gold & Platinum customers
SELECT
    customer_name,
    membership_tier,
    total_spend,
    transaction_count
FROM  public.vw_customer_spend
WHERE membership_tier IN ('Gold','Platinum')
ORDER BY total_spend DESC
LIMIT 5;

-- Results:
-- David Thompson   Platinum   $235,477   23 visits
-- Jessica Brown    Gold       $230,901   20 visits
-- David Martin     Gold       $219,006   19 visits
-- Jennifer Lopez   Gold       $218,078   18 visits
-- Mary Gonzalez    Platinum   $215,282   20 visits
```

Views vs. CTAS: Use a view when you always want fresh data. Use CREATE TABLE AS SELECT (CTAS) to snapshot or cache a slow query for speed.

Views: Pros, Cons & Yellowbrick Gotchas

✓ Simple to create

One SQL statement. No storage, no scheduling, always fresh. Any analyst with CREATE privilege can write one.

Use views freely for standard joins and business rules.

✓ Centralised logic

A business rule defined in a view is identical for every analyst. Fix it once — fixed everywhere.

Put shared definitions (e.g. 'active customer', 'completed sale') in a view.

✗ No parameters

A view is a fixed query. You cannot pass in a filter value. For parameterised logic, use a Stored Procedure.

Add WHERE when querying — or use a stored procedure for runtime parameters.

✗ Re-executes every query

Views store no data. Every reference reruns the underlying SQL. A complex nested view can be slow.

For speed, CTAS to materialise the result into a real table.

⚠ Nested view ordering

Yellowbrick reconstructs join order when views reference other views. This can produce unexpected slow plans.

Avoid deeply nested views. If slow, inline the inner view SQL directly.

⚠ Predicate pushdown limits

Filters on computed column expressions (e.g. WHERE full_name = 'Alice') may not push efficiently to the scan.

Filter on original base-table columns whenever possible.

Yellowbrick

Stored Procedures

What is a Stored Procedure?

A stored procedure is named, saved code — like a view, but it can accept parameters, contain logic, and run multi-step operations.

- A view is a saved question — returns data but accepts no input
- A procedure is a saved workflow — accepts parameters, runs code, can INSERT/UPDATE/DELETE
- Written in PL/pgSQL — Yellowbrick's procedural language (similar to PostgreSQL 9.5)
- Runs on demand: CALL it or SELECT from it, and it executes once
- Stored in the database — shared across all users with EXECUTE privilege

Feature	View	Stored Proc
Parameters?	✗ No	✓ Yes
Stores data?	✗ No	✗ No
Multi-step logic?	✗ No	✓ Yes
IF/THEN branching?	✗ No	✓ Yes
Easy to debug?	✓ Yes	Harder
Called how?	SELECT	CALL/SELECT
Performance	Re-executes	Re-executes
Best for	Joins, rules	ETL, params
Language needed	SQL only	PL/pgSQL

Views vs. Stored Procedures

A view stores a query. A procedure stores logic. Use a view when anyone needs the same joined data. Use a procedure when you need parameters, branching (IF/THEN), or multi-step operations.

How Yellowbrick Executes Stored Procedures

Understanding where code runs explains why certain patterns are fast — and why others are dangerous.

Manager Node (Frontend)

Where PL/pgSQL runs

- All procedural code runs here
- IF/THEN, loops, DECLARE variables
- RAISE INFO messages processed here
- Procedure call overhead is here

```
-- This code runs on the
-- manager (frontend):
FOR v_row IN ...
  RETURN NEXT v_row;
END LOOP;
```

SQL Inside Procedures

Dispatched to workers

- SELECT/INSERT/UPDATE inside procs
- Sent to compute workers as normal
- Full MPP parallelism — same speed
- Aggregation and joins use all nodes

```
-- This SQL dispatches to
-- all compute workers:
SELECT p.category,
  SUM(f.total)
FROM fact_sales f ...
```

⚠ The Danger Zone

Row-by-row cursor loops

- CURSOR loop processes 1 row at a time
- Each row: manager processes + calls back
- Saturates the manager node
- Think: 1M iterations = 1M manager calls

```
-- NEVER do this in YB:
FOR rec IN SELECT * FROM
    big_table
LOOP
  UPDATE ... WHERE
id=rec.id;
END LOOP; -- SLOW!
```

✓ The Right Pattern

Set-based operations

- Do all work in one SQL statement
- Let the workers do the heavy lifting
- Single SQL replaces any loop
- Manager just coordinates, workers execute

```
-- DO THIS instead:
INSERT INTO summary_tbl
SELECT category,
  SUM(total) AS revenue
FROM fact_sales f
JOIN dim_product p ...
GROUP BY category;
```

Stored Procedure: Basic Syntax

Yellowbrick uses PL/pgSQL — PostgreSQL 9.5's procedural language. Only plpgsql is supported.

Syntax skeleton

```
CREATE [ OR REPLACE ] PROCEDURE proc_name
    ( [ arg_name  arg_type  [ DEFAULT expr ] ] )
    [ RETURNS return_type ]
    LANGUAGE plpgsql
AS $$
[ DECLARE
    var_name  type  [ := default_value ];
]
BEGIN
    -- your PL/pgSQL code here
    -- SQL statements, IF/THEN, loops
END;
$$
```

Key Points

1 `OR REPLACE`

Redefines the procedure in-place. Existing callers are not broken.

2 `DECLARE block`

Optional. Declare local variables before BEGIN. Variables are session-scoped to the call.

3 `$$ delimiters`

The $$ markers wrap the procedure body. You can also use $BODY$ or any $label$.

4 `LANGUAGE plpgsql`

Required. Yellowbrick only supports plpgsql — not SQL, Python, or other languages.

Return Types: What a Stored Procedure Can Return

Yellowbrick supports three return types. The return type determines how you must call the procedure.

Return Type	Call with	Best for	Example use case
VOID	CALL proc()	Actions with no result: INSERTs, UPDATEs, logging	CALL show_category_summary('Electronics')
Scalar (single value)	SELECT proc()	Return one computed value: a count, a total, a flag	SELECT COUNT(*) -- or call a proc returning 1 number
SETOF table_type	SELECT * FROM proc()	Return a result set shaped like an existing table	SELECT * FROM get_top_customers(10)

CALL vs. SELECT

- CALL: use for VOID procs. The procedure executes; nothing is returned.
- SELECT: use for scalar or SETOF procs. Results are returned as query output.
- **Calling a SETOF proc with CALL returns an error. This is a common gotcha.**

SETOF Requirement

- RETURNS SETOF requires a matching table that defines the row shape.
- Create a persistent or TEMP table first — the procedure will RETURNS SETOF that table.
- Cannot RETURN an inline row type (no RETURN TABLE support). Must match an existing table definition.

Example 1: VOID Procedure — Category Summary

VOID procedures perform actions but return no rows. Use CALL. Use RAISE INFO for feedback during testing.

show_category_summary(p_category)

```
CREATE OR REPLACE PROCEDURE
    public.show_category_summary(
        p_category VARCHAR(50))
    RETURNS VOID LANGUAGE plpgsql
AS $$
DECLARE
    v_items   BIGINT;
    v_units   BIGINT;
    v_revenue NUMERIC(15,2);
BEGIN
    SELECT COUNT(*),
           SUM(f.quantity),
           ROUND(SUM(f.total), 2)
    INTO   v_items, v_units, v_revenue
    FROM   samples.retail.fact_sales f
    JOIN   samples.retail.dim_product p
      ON   f.product_key = p.product_key
    WHERE  p.category = p_category;

    RAISE INFO 'Category:   %', p_category;
    RAISE INFO 'Line Items: %', v_items;
    RAISE INFO 'Units Sold: %', v_units;
    RAISE INFO 'Revenue:    $%', v_revenue;
END; $$
```

Calling and testing it

```
-- CALL (VOID proc):
CALL public.show_category_summary(
    'Electronics');

-- RAISE INFO output:
-- INFO: Category:   Electronics
-- INFO: Line Items: 274606
-- INFO: Units Sold: 824000
-- INFO: Revenue:    $442008997.69

-- Try other categories:
CALL public.show_category_summary(
    'Furniture');
CALL public.show_category_summary(
    'Clothing');
```

When to use VOID

VOID procs are ideal for ETL steps, data quality checks, and logging. RAISE INFO prints progress to your SQL client during execution.

Example 2: SETOF Procedure — Top Customers

get_top_customers(p_limit INT) — returns table rows

SETOF procedures return rows shaped like an existing table. Call with SELECT * FROM proc().

```
-- Step 1: Create return-type table
CREATE TABLE IF NOT EXISTS
    public.top_customer_result (
    customer_name   VARCHAR(100),
    membership_tier VARCHAR(20),
    total_spend     NUMERIC(15,2),
    visits          BIGINT
) DISTRIBUTE ON (customer_name);

-- Step 2: Create the procedure
CREATE OR REPLACE PROCEDURE
  public.get_top_customers(p_limit INT)
  RETURNS SETOF
      public.top_customer_result
  LANGUAGE plpgsql
AS $$
DECLARE
  v_row top_customer_result%ROWTYPE;
  v_sql VARCHAR(500) = '
    SELECT
      c.first_name||'' ''||c.last_name,
      c.membership_tier,
      ROUND(SUM(f.total), 2),
      COUNT(DISTINCT
          f.transaction_id)
    FROM samples.retail.fact_sales f
    JOIN samples.retail.dim_customer c
      ON f.customer_key=c.customer_key
    GROUP BY 1,2 ORDER BY 3 DESC
    LIMIT $1';
BEGIN
  FOR v_row IN EXECUTE(v_sql)
              USING p_limit
  LOOP RETURN NEXT v_row; END LOOP;
END; $$
```

Calling and output

Use SELECT * FROM, not CALL. Calling a SETOF proc with CALL = error.

```
-- SELECT * FROM (not CALL):
SELECT * FROM
    public.get_top_customers(5);

-- Result:
-- customer_name    |tier  |spend     |visits
-- Mary Thomas      |Bronze|10,067,563|1134
-- Karen Lee        |Bronze|10,057,674|1151
-- Barbara Hernandez|Bronze|10,032,301|1133
-- Charles Smith    |Bronze| 9,939,483|1139
-- Linda Jackson    |Bronze| 9,887,664|1143
```

How SETOF works

%ROWTYPE declares a variable matching the table structure. RETURN NEXT appends one row. FOR…LOOP iterates the SQL results and returns them one at a time.

Arguments, Defaults & Overloading

Same procedure name, different signatures — Yellowbrick routes to the matching version.

Overloaded: adding an optional tier filter

```
-- Overloaded: add optional tier filter
CREATE OR REPLACE PROCEDURE
  public.get_top_customers(
      p_limit INT,
      p_tier  VARCHAR(20))
  RETURNS SETOF
      public.top_customer_result
  LANGUAGE plpgsql
AS $$
DECLARE
  v_row top_customer_result%ROWTYPE;
  v_sql VARCHAR(500) = '
    SELECT c.first_name||'''' ''''
           ||c.last_name,
      c.membership_tier,
      ROUND(SUM(f.total),2),
      COUNT(DISTINCT f.transaction_id)
    FROM samples.retail.fact_sales f
    JOIN samples.retail.dim_customer c
      ON f.customer_key=c.customer_key
    WHERE c.membership_tier = $2
    GROUP BY 1,2 ORDER BY 3 DESC
    LIMIT $1';
BEGIN
  FOR v_row IN EXECUTE(v_sql)
    USING p_limit, p_tier
  LOOP RETURN NEXT v_row; END LOOP;
END; $$
```

Calling both versions

```
-- Same name, different signatures:

-- Version 1 (no tier filter):
SELECT * FROM get_top_customers(5);

-- Version 2 (with tier filter):
SELECT * FROM
  get_top_customers(5, 'Platinum');

-- Result for Platinum tier:
-- Elizabeth Taylor  1,972,058  211
-- Jessica Garcia    1,678,662  189
-- Lisa Perez        1,656,665  192

-- Integer downcast gotcha:
-- FAILS: get_top_customers(5.0, 'Gold')
-- Fix: cast explicitly
SELECT * FROM
  get_top_customers(5::INT, 'Gold');
```

Integer downcast: YB does not implicitly downcast integers. Literal 5 is INTEGER; if proc expects SMALLINT, cast explicitly: 5::SMALLINT.

TEMP Tables Inside Stored Procedures

Always use ON COMMIT DROP

Without ON COMMIT DROP, temp tables persist across calls and cause 'table already exists' errors.

```
-- GOOD: ON COMMIT DROP
CREATE OR REPLACE PROCEDURE
    public.process_daily_sales()
    RETURNS VOID LANGUAGE plpgsql
AS $$
BEGIN
    DROP TABLE IF EXISTS tmp_sales;

    CREATE TEMP TABLE tmp_sales (
        category   VARCHAR(50),
        revenue    NUMERIC(15,2)
    ) ON COMMIT DROP;

    INSERT INTO tmp_sales
    SELECT p.category, SUM(f.total)
    FROM   samples.retail.fact_sales f
    JOIN   samples.retail.dim_product p
      ON   f.product_key = p.product_key
    GROUP BY p.category;

    -- ... further logic ...
END; $$
```

What goes wrong without it

The BAD pattern and its consequences:

```
-- BAD: Missing ON COMMIT DROP
BEGIN
    CREATE TEMP TABLE tmp_sales (
        category VARCHAR(50),
        revenue  NUMERIC(15,2)
    );  -- NO ON COMMIT DROP!

    INSERT INTO tmp_sales ...
END; $$

-- Consequences:
-- 1st call: runs fine
-- 2nd call: ERROR - table exists!

-- If used as SETOF return type:
--   Procedure drops when session
--   disconnects (temp table drops,
--   proc reference becomes invalid)

-- Rule: NEVER use TEMP table as
--   SETOF return type. Use a
--   permanent table instead.
```

Transaction Handling in Stored Procedures

Procedures called via CALL run in a single parent transaction. COMMIT and ROLLBACK are available — but with important side effects.

COMMIT and ROLLBACK inside CALL

```
CREATE OR REPLACE PROCEDURE
    public.load_with_commits()
    RETURNS VOID LANGUAGE plpgsql
AS $$
BEGIN
    -- Step 1: insert and commit
    INSERT INTO stage_tbl ...
    COMMIT;  -- releases locks

    -- Step 2: transform and commit
    INSERT INTO mart_tbl
    SELECT ... FROM stage_tbl;
    COMMIT;
END; $$
```

Locking gotchas to watch for

- TRUNCATE inside a proc holds an exclusive lock for the entire transaction unless you COMMIT immediately after
- Any table scanned during the proc holds a non-exclusive read lock until the proc finishes or you COMMIT
- An open transaction blocks garbage collection (GC) for subsequent transactions
- Long-running procs can block other users — keep procedures short and commit intermediate steps
- COMMIT/ROLLBACK only works when called via CALL. When called via SELECT, transaction control is handled by the caller

Best practice

Best practice: use explicit COMMIT after each major DML step in long ETL procedures. This releases locks sooner, allows GC to run, and prevents one failure from rolling back hours of work.

Yellowbrick Stored Procedure Limitations

Know these before you build. Most are not bugs — they reflect the architecture.

✗ plpgsql only

No other languages (SQL, Python, JavaScript). The LANGUAGE clause must always be plpgsql.

✗ No triggers

Yellowbrick does not support trigger procedures. Procedures must be called explicitly.

✗ No SECURITY DEFINER

Cannot run a procedure with the owner's privileges. All procs execute with the caller's privileges.

✗ No RETURN TABLE

Cannot define the return row type inline. Must reference an existing table or view.

✗ No nested CALL

You cannot CALL a nested procedure from inside another procedure. Use SELECT to invoke nested procs.

⚠ Frontend execution (manager)

PL/pgSQL logic runs on the manager node. SQL inside dispatches to workers. Row-by-row cursor loops saturate the manager — always prefer set-based SQL inside procedures.

Stored Procedures: Pros & Cons

Stored procedures are powerful — but they come with real trade-offs. Use them for the right tasks.

Pros — when stored procedures shine

- Parameters — filter, paginate, and customise results at call time
- Multi-step logic — chain INSERTs, UPDATEs, and SELECTs in one call
- Reusable ETL — write the pipeline once, run it on any schedule
- Centralised business rules — one proc, consistent behaviour
- Access control — grant EXECUTE without granting table access
- Overloading — same name, different signatures for flexible calling

Cons — when to think twice

- Frontend execution — procedural logic runs on the manager, not workers
- Cursor loops are dangerous — row-by-row iteration saturates the manager node
- Harder to debug — no query plan, limited visibility in sys.log_query
- Limited return types — cannot return inline row types or arrays
- No SECURITY DEFINER — cannot safely escalate privileges inside a proc
- Temp table gotchas — ON COMMIT DROP required; SETOF on temp tables drops the proc

Rule of thumb

Rule of thumb: if it can be done in a single SQL statement, do it in SQL. Use a stored procedure when you genuinely need parameters, branching logic, or multi-step operations that cannot be expressed as one query.

Stored Procedures - Optimization Tips

Front-end orchestration vs back-end set processing — and where filters actually run.

Filtering a Procedure's Output

A procedure is an orchestration layer; the set work belongs in the backend query it runs.

PROBLEM

- SELECT ... FROM my_proc() WHERE ... LIMIT 10 is slow.
- Manager node memory/CPU spikes on proc calls.
- Whole result set returns before your filter applies.

WHY IT HAPPENS

- A procedure's results are processed in the FRONT END (manager node).
- There is no predicate pushdown, LIMIT, or column elimination through a procedure.
- The entire result set is materialised on the manager before WHERE/LIMIT run.

HOW TO FIX

- Push the predicate INTO the procedure's backend query (e.g. via an argument).
- Don't rely on filtering a procedure's output from the outside.
- Return only the rows the caller needs.

Set-Based, Not Row-by-Row, in Procedures

Classic OLTP patterns ported straight from Oracle/SQL Server are the usual culprit.

OFFENDING SQL

```
-- OLTP-style cursor loop:
FOR r IN SELECT id FROM staging LOOP
  UPDATE target
     SET amt = r.amt
   WHERE id = r.id;   -- per row!
END LOOP;
```

Per-row loops saturate the single manager node instead of spreading work across workers.

FIXED SQL

```
-- one set-based statement
UPDATE target t
SET    amt = s.amt
FROM   staging s
WHERE  t.id = s.id;

-- and push filters into the backend
-- query, not onto proc output
```

One set-based statement runs across all workers in parallel; no manager-node row loop.

Takeaway: Write procedures around set-based statements; reserve loops for control flow, never for data movement.

Yellowbrick

From SAS to Yellowbrick

The Mindset Shift: SAS vs. Yellowbrick SQL

SAS processes datasets row-by-row. SQL processes entire sets at once. This is the #1 migration concept.

SAS — iterates row by row

```
/* SAS: row-by-row DATA step */
data rev_summary;
  set transactions;
  by category;
  if first.category then rev=0;
  rev + amount;
  if last.category then output;
run;

/* PROC MEANS */
proc means data=transactions sum;
  class category; var amount;
run;

/* Row-by-row update loop */
data _null_;
  set updates;
  call execute(
    'UPDATE sales SET '
    ||'status=''done'' '
    ||'WHERE id='||id||';');
run;
/* 1M rows = 1M SQL statements */
```

Yellowbrick SQL — set-based

```
-- SQL: entire set at once
SELECT
    p.category,
    SUM(f.total)   AS total_revenue,
    COUNT(*)       AS line_items,
    AVG(f.total)   AS avg_sale
FROM samples.retail.fact_sales f
JOIN samples.retail.dim_product p
  ON f.product_key = p.product_key
GROUP BY p.category
ORDER BY total_revenue DESC;

-- Set-based update (no loop!):
UPDATE sales_tbl s
SET    status = 'done'
FROM   updates_tbl u
WHERE  s.id = u.id;
```

Key rule

If you find yourself writing a loop that fires one SQL per row, stop. Rewrite as a single set-based operation. This is the #1 performance fix in SAS migration.

SAS to Yellowbrick: Pattern Map

Most SAS constructs map directly to SQL. Once you think in sets, the translation becomes natural.

SAS Construct	Yellowbrick Equivalent	Notes
DATA step (transform)	INSERT INTO ... SELECT ...	Replace row logic with set-based ops. CTAS for snapshots.
DATA step (create dataset)	CREATE TABLE AS SELECT	CTAS creates a permanent table from a query.
PROC MEANS / SUMMARY	SELECT ... GROUP BY + aggregates	SUM, AVG, MIN, MAX, COUNT work identically.
PROC SORT	ORDER BY (or SORT ON at load)	SORT ON in table DDL + ORDER BY on INSERT for sorted storage.
PROC FREQ	SELECT col, COUNT(*) GROUP BY col	Add window function for % of total.
PROC TRANSPOSE	CASE WHEN	Conditional aggregation for pivot; pandas for complex transpose.
SAS Macro (no params)	View	Standard join/report macro → CREATE OR REPLACE VIEW.
SAS Macro (with params)	Stored Procedure	Parameterised macro → CREATE PROCEDURE ... (p_x TYPE).
PROC SQL pass-through	Direct Yellowbrick SQL	Most PROC SQL syntax works with minor adjustments.

SAS Macros: View or Stored Procedure?

The type of SAS macro determines which Yellowbrick tool replaces it. Apply this rule consistently.

Macro (no params) → View

- Standard join or report
- No parameter inputs
- Multiple teams query it

```
%macro revenue;
proc sql;
  select category,
  sum(amount) from t
  group by 1;
quit; %mend;
-- Yellowbrick:
CREATE OR REPLACE VIEW
  public.vw_revenue AS
SELECT p.category,
  SUM(f.total) AS rev
FROM fact_sales f
JOIN dim_product p
  ON ...
GROUP BY p.category;
```

Macro (with params) → Proc

- Accepts %let parameters
- Filters or customises query
- Different call each time

```
%macro top(tier);
proc sql;
  select * from custs
  where tier=&tier;
quit; %mend;
-- Yellowbrick:
CREATE PROCEDURE
  get_top_customers(
    p_tier VARCHAR(20))
  RETURNS SETOF ...
SELECT * FROM
  get_top_customers(
    'Platinum');
```

PROC SQL → Direct SQL

- Most PROC SQL is valid SQL
- No wrapper needed
- Run directly in Yellowbrick

```
proc sql;
  select t.*,c.name
  from trans t
  join customers c
  on t.id=c.id;
quit;
-- Yellowbrick:
SELECT t.*, c.name
FROM   trans t
JOIN   customers c
  ON t.id = c.id;
```

Row Loop → Set-based SQL

- Row-by-row DATA step
- IF/THEN per row
- Iterative UPDATE per row

```
data work; set src;
  if status='P' then
    rev=amount*0.9;
  else rev=amount;
run;
-- SQL (1 statement):
SELECT amount *
  CASE WHEN status='P'
       THEN 0.9
       ELSE 1.0
  END AS revenue
FROM src;
```

Anti-Pattern: The Cursor Loop

The most common SAS migration mistake: replacing a DATA step loop with a cursor loop in a stored procedure.

BAD: cursor loop (SAS-style iteration)

```
/* BAD: Cursor loop in stored proc */
CREATE OR REPLACE PROCEDURE
    public.update_status_loop()
    RETURNS VOID LANGUAGE plpgsql
AS $$
DECLARE
    v_id    BIGINT;
    v_cur   CURSOR FOR
        SELECT sales_key
        FROM   samples.retail.fact_sales
        WHERE  payment_status='pending';
BEGIN
    OPEN v_cur;
    LOOP
        FETCH v_cur INTO v_id;
        EXIT WHEN NOT FOUND;
        -- One UPDATE per row!
        UPDATE fact_sales_staging
        SET    status = 'reviewed'
        WHERE  sales_key = v_id;
    END LOOP;
    CLOSE v_cur;
END; $$
-- 274k rows = 274k round-trips!
```

GOOD: set-based rewrite

```
/* GOOD: Set-based rewrite */
CREATE OR REPLACE PROCEDURE
    public.update_status_set()
    RETURNS VOID LANGUAGE plpgsql
AS $$
BEGIN
    -- Single UPDATE for all rows
    UPDATE fact_sales_staging s
    SET    status = 'reviewed'
    FROM   samples.retail.fact_sales f
    WHERE  s.sales_key = f.sales_key
      AND  f.payment_status = 'pending';
END; $$

-- One statement.
-- All compute workers used.
```

Why this matters

Cursor loops saturate the manager node — they were designed for OLTP systems, not MPP data warehouses. Any loop that fires one SQL per row should be rewritten as a single set-based statement.

SAS Migration Checklist

Work through this checklist when converting SAS programs to Yellowbrick SQL.

① Identify all row-by-row loops

Find every DATA step loop and cursor. Each one needs to be rewritten as a set-based SQL operation.

Look for: do/end blocks, iterating MERGE, row-by-row UPDATE/DELETE

② Replace macros with views or procs

Macro with no params → CREATE OR REPLACE VIEW. Macro with params → CREATE OR REPLACE PROCEDURE.

Map each %macro to its Yellowbrick equivalent using the pattern table from slide 21.

③ Validate row counts before and after

After migrating each piece of logic, compare row counts and key aggregates between SAS and SQL output.

SELECT COUNT(*), SUM(total), MIN(date), MAX(date) FROM both versions.

④ Work in your personal schema

Create all migration objects in your personal schema (e.g. msuarez.*) until validated. Move to shared schemas after sign-off.

Use: CREATE VIEW msuarez.vw_sales_test AS ... — keep shared schemas clean.

⑤ Test on a data slice first

Add WHERE year = 2025 or LIMIT 10000 while developing. Validate logic before running on full dataset.

Add LIMIT to stored procedure SQL during dev: ... ORDER BY 3 DESC LIMIT 1000

⑥ Use RAISE INFO for debugging

Inside stored procedures, use RAISE INFO to print variable values and progress messages during testing.

RAISE INFO 'Processing category: %, rows: %', p_category, v_count;

Yellowbrick

ETL with ybsql Variables

Why Drive ETL from ybsql?

ybsql is Yellowbrick's command-line client. With variables and meta-commands it becomes a lightweight ETL orchestrator — one .sql script that runs a parameterized series of transformations.

One script, many steps

Chain staging → transform → load as ordered SQL statements in a single file run with -f.

Parameterized & repeatable

Variables let the same script run for any date, schema, or batch — no copy-paste edits.

Nothing to deploy

No database object to create or maintain. The logic lives in a versioned .sql file.

A minimal run

```
# Run a script against Yellowbrick
ybsql -h dw.prod -d sales -U etl \
      -f daily_etl.sql

# -f runs the file; meta-commands
# (\set, \gset, \echo) orchestrate
# the steps inside it.
```

When to choose ybsql

Reach for ybsql when the work is pure SQL run on a schedule.

Prefer a stored procedure when logic must live in the database and be CALLed by many clients; prefer Python when you need data-frame logic, APIs, or charts.

Setting Variables: \set and \gset

Define values once, then reference them throughout the script.

\set — assign a literal value

```
-- Static values, defined up front
\set target_schema  staging
\set load_date      '2026-06-01'
\set batch_size      50000

-- Echo a variable to check it
\echo Loading into :target_schema
```

\gset — capture a query result

```
-- Run a one-row query; each column
-- alias becomes a variable
SELECT MAX(order_date) AS max_dt,
       COUNT(*)        AS row_cnt
FROM   staging.orders
\gset

-- :max_dt and :row_cnt now exist
\echo Latest staged date: :max_dt
```

Key difference

\set assigns text you type in the script; \gset assigns values pulled from the database at run time. The query feeding \gset must return exactly one row, and its column aliases become the variable names.

Referencing Variables in SQL: :var, :'var', :"var"

How you quote the reference tells ybsql what kind of value to substitute.

:var — raw substitution

For numbers and keywords. Pasted in verbatim with no quoting.

e.g. LIMIT :batch_size

:'var' — string literal

Single quotes. For text and dates inside WHERE / VALUES.

e.g. WHERE order_date >= :'load_date'

:"var" — SQL identifier

Double quotes. For schema, table, or column names.

e.g. INSERT INTO :"target_schema".orders

Quote by value type

```
-- WRONG: string with no quotes
INSERT INTO staging.orders
SELECT * FROM raw.orders
WHERE order_date >= :load_date;  -- err

-- RIGHT: match quotes to the value
INSERT INTO :"target_schema".orders
SELECT * FROM raw.orders
WHERE  order_date >= :'load_date'
LIMIT  :batch_size;
```

Why it matters

Bare :var on a string produces unquoted text and a syntax error. Use :'var' for literals and :"var" for object names so the generated SQL is always valid.

Passing Variables In: the -v / --set Option

Supply variables from the command line so one script runs for any date, schema, or batch.

Invoke with -v

```
# One -v (or --set) per variable
ybsql -h dw.prod -d sales -U etl \
  -v load_date='2026-06-01' \
  -v target_schema=staging \
  -v batch_size=50000 \
  -f daily_etl.sql

# --set is the long form of -v
```

The script consumes them

```
-- daily_etl.sql references the
-- values passed on the command line
INSERT INTO :"target_schema".orders
SELECT *
FROM   raw.orders
WHERE  order_date = :'load_date'
LIMIT  :batch_size;

-- No edits needed between runs
```

Scheduling tip

In cron or a job scheduler, compute the date and inject it at run time: ybsql -v load_date="$(date +%F)" -f daily_etl.sql . One script, any run date — driven entirely by the values you pass.

Error Handling & Status Output

Stop on the first failure, and make each step report what it did.

Fail fast with ON_ERROR_STOP

```
-- Abort on first error and return a
-- non-zero exit code to the shell
\set ON_ERROR_STOP on

\echo Step 1: staging load
INSERT INTO staging.orders ... ;

\echo Step 2: transform
INSERT INTO mart.orders ... ;
```

Report status & timing

```
\timing on            -- ms per step
-- \echo  -> stderr (script trace)
-- \qecho -> query output stream

-- Capture a row count for the log
SELECT COUNT(*) AS loaded
FROM   mart.orders
WHERE  load_date = :'load_date'
\gset
\qecho Rows loaded today: :loaded
```

Exit codes drive your scheduler

With ON_ERROR_STOP on, ybsql exits non-zero the moment a statement fails. Cron, Airflow, or a shell wrapper can branch on $? to alert or retry. Add \set ECHO queries to log every statement as it runs.

Transactions in ybsql Scripts: COMMIT & ROLLBACK

ybsql runs autocommit by default — wrap steps in a transaction so a multi-step load succeeds or fails as a unit.

Explicit BEGIN / COMMIT

```
-- BEGIN groups steps into one unit
\set ON_ERROR_STOP on
BEGIN;
  INSERT INTO mart.orders
  SELECT * FROM staging.orders;

  UPDATE mart.daily_summary
  SET    loaded = true;
COMMIT;

-- On error: the script aborts and
-- the open transaction rolls back
```

Turn off autocommit

```
-- Open an implicit transaction
\set AUTOCOMMIT off
\set ON_ERROR_STOP on

INSERT INTO mart.orders ... ;
UPDATE mart.daily_summary ... ;

COMMIT;          -- or ROLLBACK;

-- \set ON_ERROR_ROLLBACK on wraps
-- each statement in a savepoint
```

Atomic ETL

Combine BEGIN/COMMIT with ON_ERROR_STOP so a multi-step load either fully commits or leaves the target unchanged. Use ROLLBACK to discard a test run; set ON_ERROR_ROLLBACK on to keep the transaction alive when a single statement fails.

Putting It Together: A Multi-Step ETL Script

One parameterized daily_etl.sql — staging → transform → verify — with fail-fast and logging.

daily_etl.sql

```
-- Params (pass with -v): load_date, target_schema
\set ON_ERROR_STOP on
\timing on
\echo ===== ETL start: :load_date =====

-- 1. Stage raw rows for the date
\echo Step 1/3  staging
INSERT INTO :"target_schema".stg_orders
SELECT * FROM raw.orders
WHERE  order_date = :'load_date';

-- 2. Transform / clean
\echo Step 2/3  transform
INSERT INTO :"target_schema".orders
SELECT order_id, customer_id,
       UPPER(status), total
FROM   :"target_schema".stg_orders;

-- 3. Verify & log the row count
\echo Step 3/3  verify
SELECT COUNT(*) AS loaded
FROM   :"target_schema".orders
WHERE  order_date = :'load_date'
\gset
\qecho Loaded :loaded rows for :load_date
\echo ===== ETL done =====
```

Run it

```
ybsql -h dw.prod -d sales -U etl \
  -v load_date='2026-06-17' \
  -v target_schema=staging \
  -f daily_etl.sql
echo "exit code: $?"
```

Every concept in one file

-v inputs supply the run date & schema.
:'…' quotes literals, :"…" quotes object names.
ON_ERROR_STOP gives fail-fast + exit codes.
\gset + \qecho capture and log row counts.

ybsql Scripts vs. Stored Procedures

Two ways to package reusable logic in Yellowbrick — choose by where the logic should live and how it gets called.

Dimension	ybsql script (.sql)	Stored procedure
Where logic lives	Client-side .sql file — versioned like code	Compiled object stored in the database
Parameters	:vars via \set, \gset and -v	Typed IN arguments, with defaults
How you run it	ybsql -f daily_etl.sql	CALL proc_name(args)
Procedural logic	Linear SQL only — no loops or IF	Full PL/pgSQL: loops, IF, variables
Error handling	BEGIN/COMMIT/ROLLBACK + ON_ERROR_STOP	BEGIN…EXCEPTION handlers + COMMIT/ROLLBACK
Deployment	Nothing to deploy — just the file	CREATE PROCEDURE; manage grants
Best fit	Scheduled, set-based ETL pipelines	Reusable logic many clients CALL

Rule of thumb

Use ybsql when the work is a scheduled sequence of set-based SQL you want in version control; reach for a stored procedure when the logic is reusable, needs procedural control flow, or must be callable by many clients from inside the database.

Yellowbrick

Python for Analytics

SAS to Python: Library Reference

Every SAS procedure has a Python equivalent. Python connects directly to Yellowbrick via psycopg2.

SAS Procedure	Python Library / Method	What it does
DATA step / PROC SQL	`pandas + psycopg2`	Read SQL results into a DataFrame; transform with pandas operations
PROC MEANS / SUMMARY	`df.groupby().agg()`	Group-by aggregations: sum, mean, min, max, std, count
PROC FREQ	`df['col'].value_counts()`	Frequency table with counts and optional percentages
PROC SORT	`df.sort_values('col')`	Sort a DataFrame by one or more columns
PROC TRANSPOSE	`df.pivot_table() / df.melt()`	Pivot rows to columns or columns to rows
PROC REG / GLM	`statsmodels.formula.api / sklearn`	Linear regression, generalised linear models
PROC LOGISTIC	`sklearn.linear_model.LogisticRegression`	Logistic regression and classification
PROC UNIVARIATE	`scipy.stats / df.describe()`	Distribution stats: quartiles, skewness, kurtosis
SAS/GRAPH	`matplotlib.pyplot / seaborn`	Bar charts, line plots, histograms, heatmaps
ODS Output	`df.to_excel() / df.to_csv()`	Write results to Excel or CSV files

Connecting to Yellowbrick with psycopg2

psycopg2 is the standard PostgreSQL driver for Python. Yellowbrick is compatible out of the box.

Connect and query into a DataFrame

```
import psycopg2, pandas as pd

conn = psycopg2.connect(
    host     = 'your-yb-host',
    port     = 5432,
    dbname   = 'samples',
    user     = 'your_username',
    password = 'your_password',
    sslmode  = 'require')

# Aggregate in SQL — bring small
# result set to Python
sql = '''
    SELECT p.category,
           ROUND(SUM(f.total),2)
                        AS revenue,
           COUNT(DISTINCT
             f.transaction_id) AS txns
    FROM retail.fact_sales f
    JOIN retail.dim_product p
      ON f.product_key=p.product_key
    GROUP BY p.category
    ORDER BY revenue DESC
'''
df = pd.read_sql(sql, conn)
conn.close()
print(df.head())
```

Key Points

1 Do the work in SQL

Let Yellowbrick's workers handle aggregation and joins. Bring a summarised result to Python — not millions of raw rows.

2 Install psycopg2

pip install psycopg2-binary. Includes the C driver — no separate PostgreSQL install needed.

3 Use pd.read_sql()

Reads query results directly into a pandas DataFrame. Column types handled automatically.

4 Close connections

Always call conn.close() when done. Open connections hold locks and consume server resources.

Using Environment Variables to Connect in Python

How to grab environment variables:

Setup Environment

```
ubuntu@ip-172-31-43-233 /mnt/perftestv3/melco (0.24s)
cat melco.env
export YBHOST=af5c856210e144a7e8d2f8652ad72760-43b9333ed7bb49a4.elb.us-east-1.amazonaws.com
export YBUSER=msuarez
export YBDATABASE=msuarez
export YBPASSWORD=[MySuperSecretPasword]

ubuntu@ip-172-31-43-233 /mnt/perftestv3/melco (0.244s)
source melco.env

ubuntu@ip-172-31-43-233  /mnt/perftestv3/melco
```

Grab connect string from environment in Python

```python
def get_connection():
    return psycopg2.connect(
        host=os.environ["YBHOST"],
        user=os.environ["YBUSER"],
        dbname=os.environ["YBDATABASE"],
        password=os.environ["YBPASSWORD"],
        port=os.environ.get("YBPORT", "5432"),
    )
```

Pandas in Action: PROC MEANS & ODS Equivalent

Pull aggregated data from Yellowbrick with SQL, then use pandas for statistics, filtering, and export.

Connect, query, and summarise

```
import psycopg2, pandas as pd

conn = psycopg2.connect(
    host='your-yb-host',
    dbname='samples', ...)

sql = '''
    SELECT d.year,
           d.month_number,
           d.month_name,
           COUNT(DISTINCT
             f.transaction_id) AS txns,
           ROUND(SUM(f.total),2)
                        AS revenue
    FROM retail.fact_sales f
    JOIN retail.dim_date d
      ON f.date_key = d.date_key
    WHERE d.year = 2026
    GROUP BY 1,2,3 ORDER BY 1,2
'''
df = pd.read_sql(sql, conn)
conn.close()

# PROC MEANS equivalent:
print(df['revenue'].describe())

# Quarterly summary:
df['q']=((df['month_number']-1)//3)+1
print(df.groupby('q')['revenue'].sum())
```

Results and further analysis

```
# describe() output:
# count    5.000000
# mean  698312382.22
# std    37698411.54
# min   663186938.78
# 50%   733972585.93
# max   736763502.41

# Quarterly totals:
# q=1: 2,133,923,027
# q=2:   711,800,393

# Filter (like SAS WHERE):
top = df.nlargest(5, 'revenue')
print(top[['month_name','revenue']])

# Rename (like SAS RENAME=):
df=df.rename(columns={
    'month_name':'Month',
    'revenue':'Revenue ($)'})

# Export to Excel (like ODS):
df.to_excel('monthly.xlsx',
            index=False)
# Export to CSV:
df.to_csv('monthly.csv',
          index=False)
```

Visualisation: SAS/GRAPH Equivalent in Python

matplotlib and seaborn replace SAS/GRAPH and ODS Graphics. Pull from Yellowbrick, plot in Python.

Revenue by category — bar chart

```
import matplotlib.pyplot as plt
import pandas as pd, psycopg2
conn = psycopg2.connect(...)
sql = '''
    SELECT p.category,
           ROUND(SUM(f.total)/1e6,1)
                  AS revenue_m
    FROM retail.fact_sales f
    JOIN retail.dim_product p
      ON f.product_key=p.product_key
    GROUP BY p.category
    ORDER BY revenue_m DESC
'''
df = pd.read_sql(sql, conn)
conn.close()

# Bar chart (like PROC GCHART):
fig, ax = plt.subplots(figsize=(10,6))
ax.barh(df['category'],
        df['revenue_m'],
        color='#0D8FA4')
ax.set_xlabel('Revenue ($M)')
ax.set_title('Revenue by Category')
plt.show()
```

What this replaces in SAS

- PROC GCHART / PROC GPLOT → matplotlib.pyplot.bar(), plot(), scatter()
- PROC SGPLOT → seaborn (sns.barplot, sns.lineplot, sns.heatmap)
- ODS Graphics → matplotlib figures saved via plt.savefig()
- SAS/GRAPH PROC PIE → pandas .plot.pie() or matplotlib
- SAS titles → ax.set_title(), set_xlabel(), set_ylabel()
- ODS to HTML → df.to_html() or Jupyter notebook output

Jupyter tip

In Jupyter notebooks, plt.show() renders charts inline. This is the Python equivalent of ODS HTML output — right in the notebook.

When to Use Each Tool

Match the tool to the job — these are complementary, not competing. The next slide shows how they fit together.

View	ybsql script	Stored Procedure	Python + psycopg2
• No parameters needed • Many people query the same join • You want fresh data every time • Logic rarely changes • Access control via SELECT grant	• Scheduled, set-based ETL in pure SQL • Parameterise with \set, \gset and -v • Steps wrapped in one transaction • Versioned .sql file — nothing to deploy • Run from cron or a job scheduler	• Logic must live in the database • Many clients CALL the same routine • Needs branching or loops (IF, FOR) • Parameterised with typed arguments • Non-analysts trigger it on demand	• Need statistics beyond SQL: regression, clustering • Creating charts and visualisations • Combining Yellowbrick data with other sources • Exploring data interactively in Jupyter • Exporting to Excel / formatted reports
Best for: Shared queries, standard joins, business rule definitions	**Best for:** Scheduled SQL ETL pipelines kept in version control	**Best for:** Reusable, parameterised logic called on demand	**Best for:** Statistical analysis, visualisation, ML, interactive exploration

How the Tools Fit Together

Not either/or — a production pipeline uses each tool for the stage it does best.

1 · Define

Views

Encode joins and business rules once as reusable, always-fresh SELECTs that every downstream step reads from.

2 · Transform & Load

ybsql scripts · Stored procedures

ybsql runs scheduled, set-based ETL from a versioned .sql file. Stored procedures hold reusable, parameterised logic many clients CALL on demand.

3 · Analyse & Present

Python + psycopg2

Pull the prepared results into pandas for statistics, machine learning and charts — or export to Excel and BI tools.

The big picture

Views feed clean data into a ybsql or stored-procedure ETL job; Python analyses and visualises the result. Each tool does what it does best.

Summary: Views, ybsql, Procedures & Python

Feature	View	ybsql	Stored Procedure	Python
Accepts parameters?	No	Yes (-v, :vars)	Yes (proc args)	Yes (function args)
Stores data?	No (virtual)	No	No	Yes (DataFrame/file)
Multi-step logic?	No	Yes (sequential SQL)	Yes (PL/pgSQL)	Yes
Statistical modelling?	No	No	Limited	Yes (statsmodels/sklearn)
Visualisation?	No	No	No	Yes (matplotlib/seaborn)
Runs where?	Compute workers	Client + cluster	Manager + workers	Your local machine
Easy to debug?	Yes	Yes (\echo, \timing)	Harder	Yes (Jupyter)
Shared across users?	Yes (GRANT)	Shared .sql files	Yes (EXECUTE)	Via shared .py files
Best for	Standard joins, shared rules	Scheduled SQL ETL	Parameterised ETL	Stats, viz, ML

Yellowbrick

Questions?

docs.yellowbrick.com · support.yellowbrick.com · michael.suarez@yellowbrick.com